

A Voyage to the Kernel



Part 15

Segment: 3.4, Day 14

Welcome to another segment of our voyage! In this article, we'll talk about device drivers.

Device drivers are important when it comes to topics closely related to kernel-level programming. The device driver layer is required to have an interface to all physical devices. There are three types of device drivers—character, block and network. The character and block devices are specific to the file subsystem—for example, in the case of tape drives and modems we meddle with character devices (which are to be accessed sequentially), while block devices can be accessed in any order.

Here is a sample piece of code that shows a 'cooling operation' (please refer to *thermal_sys.c*, in your kernel source, for more details and for all the steps).

```
/* sys I/F for cooling device */
#define to_cooling_device(_dev) \
    container_of(_dev, struct thermal_cooling_device, device)

static ssize_t
thermal_cooling_device_type_show(struct device *dev,
    struct device_attribute *attr, char *buf)
{
    struct thermal_cooling_device *cdev = to_cooling_
device(dev);

    return sprintf(buf, "%s\n", cdev->type);
}

static ssize_t
thermal_cooling_device_max_state_show(struct device *dev,
    struct device_attribute *attr, char *buf)
{

```

```
    struct thermal_cooling_device *cdev = to_cooling_
device(dev);

    return cdev->ops->get_max_state(cdev, buf);
}

static ssize_t
thermal_cooling_device_cur_state_show(struct device *dev,
    struct device_attribute *attr, char *buf)
{
    struct thermal_cooling_device *cdev = to_cooling_
device(dev);

    return cdev->ops->get_cur_state(cdev, buf);
}

static ssize_t
thermal_cooling_device_cur_state_store(struct device *dev,
    struct device_attribute *attr,
    const char *buf, size_t count)
{
    struct thermal_cooling_device *cdev = to_cooling_
device(dev);

    int state;
    int result;

    if (!sscanf(buf, "%d\n", &state))
        return -EINVAL;

    if (state < 0)
        return -EINVAL;

    result = cdev->ops->set_cur_state(cdev, state);
    if (result)
        return result;

    return count;
}

```